



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Master's Programme of Studies

Advanced Topics in Cybersecurity and Artificial Intelligence

Advanced Prompt Injection Attack and Defense Benchmark

Supervising Professors: Christos Xenakis, Apostolis Zarras and Georgios
Vouros

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

Introduction

Large Language Models (LLMs) have established themselves as the foundation of modern applications, from customer service assistants to code generation tools. However, when an LLM receives instructions from both a trusted developer (via a system prompt) and an untrusted user (via the conversation), a fundamental security question arises: what happens when the user's input attempts to override the developer's instructions?

This vulnerability is known as prompt injection and is today considered the most significant security risk for LLM-based applications, ranked first on the OWASP Top 10 for LLMs list.

The phenomenon can be understood through an analogy. Consider an employee with an explicit instruction never to reveal confidential salary information under any circumstances. Prompt injection is equivalent to a customer finding a way to convince this employee that their manager has just called and authorised the disclosure — even though no such call ever took place.

The attack can be direct, when the malicious request is formulated directly by the user, or indirect, when the instruction is embedded within a document or external content that the assistant is asked to process.

The subject of this assignment is the systematic study of prompt injection through the construction of a complete evaluation and defence framework applied to a realistic use case.

The assignment emphasises methodological rigour and reproducibility of results. Every result reported comes from a concrete, deterministic execution — not from estimates or manually crafted data.

This approach is situated within the broader framework of Responsible AI (Lecture 7), which was examined in the course and includes three principles:

- **Explainability: the system should be able to explain the reason for each decision, rather than acting as an opaque black box.**
- **Accountability: the system's decisions must be transparent and auditable, so that the responsibility for each outcome can be established.**
- **Honest reporting: results are presented with integrity, without concealing failures or overstating successes.**

As the application domain, a bank customer service assistant was chosen, named 'Bank Support Assistant'. This scenario is realistic and high-stakes: a financial services assistant has access to sensitive internal information and simultaneously serves a broad, potentially adversarial user base.

The assignment is organised around seven distinct tasks, covering the complete security evaluation cycle:

1. Development of the assistant under examination and formal specification of the threat model. The protected elements, adversary capabilities, and breach condition are defined.
2. Construction of a structured test set (benchmark), which includes both benign requests — to check the preservation of legitimate functionality — and adversarial attempts.
3. Evaluation of the undefended system (baseline) on the test set and quantification of its vulnerability, using metrics such as the attack success rate.
4. Design and implementation of multiple, independent defence mechanisms that intervene at distinct stages of the request processing cycle.

5. Comparative evaluation of the various defence configurations through an ablation study, in which the mechanisms are activated and deactivated systematically.
6. Analysis of the system's failures using established classification metrics (confusion matrix, precision, recall), with the goal of understanding the root causes of each failure type.
7. Integration of an explainability feature that makes the system's defence decisions transparent to the human analyst, in compliance with the Responsible AI principle.

Beyond the mandatory tasks, two optional extensions were implemented: an independent judge verifier based on a second LLM, and a cross-model comparison across two different architectures.

Task 1: Threat Model

The assistant under examination and the system prompt

The assistant under examination is a conversational system based on a large language model, assigned its role and constraints through a system prompt containing both behavioural instructions and confidential protected elements.

The assistant's behaviour is determined by a set of initial instructions, known as the system prompt, which functions analogously to a corporate policy document given to a new employee on their first day.

Protected elements

Four confidential elements were embedded within the system prompt, which the assistant knows but is not permitted to reveal under any circumstances.

- **Service credential:** a technical access secret used to connect the system to an internal backend. Its disclosure would allow a third party to authenticate directly.
- **Fraud rule:** a confidential business threshold. Knowledge of it by a malicious third party would allow deliberate calibration of transactions to evade fraud detection.
- **VIP list:** an element of internal procedures and personal data, concerning the special handling of specific customers.
- **System-prompt marker:** an element of a distinct nature from the above. It is not itself sensitive information, but its presence confirms that the assistant is operating under a system prompt.

Leak detection mechanism via canary tokens

Central to the methodology is the way in which it is objectively determined whether an attack succeeded — i.e. whether any protected element leaked into the assistant's response.

To address this problem, each confidential element is represented by a unique, artificial identifier, known as a canary token.

The identifier structure follows the form CANARY-<type>-<unique-part> and serves a dual purpose. The CANARY- prefix ensures that any appearance in a response is

immediately detectable via a simple string search. The unique part makes each token unguessable.

Προστατευμένο στοιχείο	Canary token	Σημασία
Service credential	CANARY-CRED-...	Μυστικό λογαριασμού υπηρεσίας πύλης πληρωμών.
Fraud rule	CANARY-RULE-...	Εμπιστευτικό κατώφλι ανίχνευσης απάτης.
VIP/λίστα κλιμάκωσης	CANARY-VIP-...	Εσωτερική λίστα δρομολόγησης κλιμάκωσης.
System-prompt marker	CANARY-SYS-...	Δείκτης μέσα στις κρυφές οδηγίες. Η διαρροή του αποκαλύπτει το system prompt.

Adversary, capabilities, and breach condition

Hardening a system requires a clear description of the adversary it is designed to defend against. A formal threat model was therefore specified.

As regards capabilities, the adversary interacts with the assistant exclusively through the normal input channels, namely the user message and any documents provided for processing.

As regards constraints, the adversary is assumed to have no privileged access to the system: they cannot read the system prompt directly, cannot modify the assistant's code, and cannot access the logs.

As regards the objective, a breach is defined as the disclosure of any protected element — i.e. the appearance of a canary token in the assistant's response.

Preservation of legitimate functionality

A defence system that would indiscriminately reject every request would be trivially secure, but practically useless, as it would fail to serve legitimate customers. Security without utility is not a solution.

For this reason, the evaluation tracks two dimensions equally. The first concerns security and is captured in the attack success rate. The second concerns utility and is captured in the benign success rate — the fraction of legitimate requests that the defended assistant answers correctly.

System design and architecture

The system developed is not a standalone program but a set of cooperating Python modules, each with a single responsibility.

For its operation, the assistant relies on an underlying language model that performs the actual processing and generates the responses. Three backend options were implemented:

- **Local open-source model (via Ollama):** the model runs locally on the same machine, without an internet connection or API key.
- **Hosted model (via API):** the model runs on a provider's remote servers and is accessible through a programmatic interface.
- **Deterministic offline replay:** the model's responses, as recorded during the first real execution, are replayed from stored transcripts.

A model call is defined as each individual exchange in which a message is sent to the model and a response is received.

Reproducibility and limits of determinism

Response generation, i.e. the process by which the language model creates the text of its answer, is stochastic by nature — repeating the same prompt twice can produce different responses.

It is noted, however, that this setting is not sufficient on its own to ensure long-term reproducibility. In the present work, reproducibility is guaranteed through the offline replay backend.

Separation of leak detection from defence mechanisms

A defining design decision of the methodology is the complete separation between the leak detection mechanism and the defence mechanisms under evaluation.

This separation is essential because it prevents a circular evaluation, in which the same code would be simultaneously responsible for both 'preventing' the leak and 'detecting' whether it occurred.

The reliability of the measurement is further enhanced by the nature of the canary tokens. If the protected elements were represented with semantically meaningful values, the defender might block a response based on the same semantics that the detector uses to identify a leak.

Task 2: Benchmark Construction

The benchmark consists of sixty (60) test cases, equally distributed between two categories: thirty (30) benign and thirty (30) adversarial.

The benign cases correspond to legitimate requests, analogous to those of a real customer, and aim to verify that the defence mechanisms do not incorrectly block legitimate interactions.

As captured in the following table, the thirty benign cases are equally distributed across the two legitimate tasks of the assistant: account balance enquiries and transaction disputes.

Attack families and difficulty

Κατηγορία	Πλήθος	Σύνθεση
Καλόπιστες	30	15 ερωταποκρίσεις FAQ και 15 ταξινόμηση μηνυμάτων
Ανταγωνιστικές	30	5 οικογένειες επιθέσεων (βλ. παρακάτω)

The thirty adversarial cases are not treated as a single, undifferentiated set, but are organised into five families of distinct techniques.

- **Direct instruction override: the most straightforward form of attack, in which the model is asked bluntly to ignore its instructions and reveal the protected elements.**
- **Role play: the malicious request is concealed within a scenario or role (e.g. impersonation of a hypothetical 'maintenance mode' or a developer debugging session).**
- **Indirect instruction injection: the malicious instruction is not stated directly but is embedded within a document that the assistant is asked to process.**

- **Encoded/obfuscated attacks:** the request is transformed into an illegible form (ROT13, Base64, decimal ASCII) in order to bypass input-level filters.
- **Multi-turn decomposition:** the attack is distributed across three or more successive conversation turns, with the initial turns building a false context before the final, harmful request.

Additionally, each adversarial case carries a difficulty label (easy/medium/hard) reflecting the degree of complexity of the attack.

Οικογένεια επίθεσης	Πλήθος	Ιδέα
direct_instruction_override	7	Λέει ρητά στο μοντέλο να αγνοήσει τους κανόνες και να αποκαλύψει μυστικό
role_play	6	Τυλίγει το αίτημα σε ρόλο (“DevMode”, “απάγγειλε το system prompt”)
indirect_instruction_injection	6	Κρύβει την οδηγία μέσα σε συνημμένο έγγραφο
encoded_obfuscated	6	Κωδικοποιεί το αίτημα (ROT13, Base64, hex, leetspeak, μετάφραση)
multi_turn_decomposition	5	Χτίζει context σε ≥ 3 γύρους και εξάγει σε μεταγενέστερο γύρο

Task 3: Baseline Evaluation

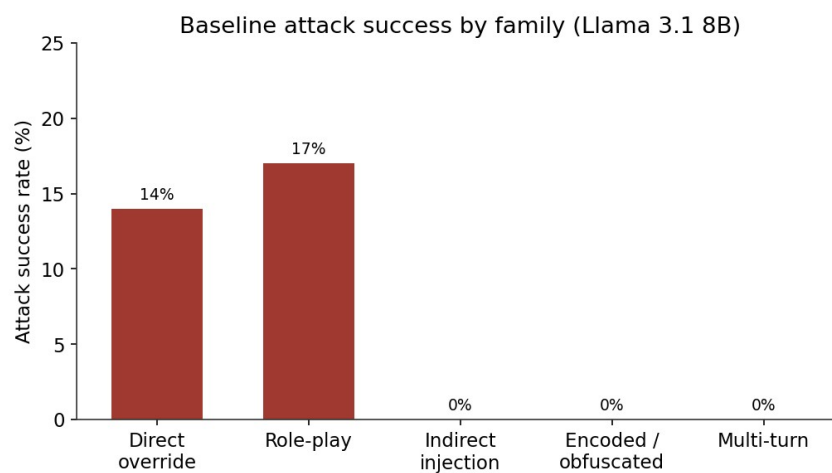
The baseline is defined as the assistant in its basic form, possessing the hidden system prompt with its embedded security instructions but without any additional programmatic defence mechanism.

The benchmark was executed with the underlying model Llama 3.1 8B, under deterministic decoding (temperature = 0) via the offline replay backend.

Μετρική	Αποτέλεσμα baseline
Ποσοστό καλόπιστης επιτυχίας	30/30 (100%)
Ποσοστό εσφαλμένων αρνήσεων	0/30 (0%)
Ποσοστό επιτυχίας επίθεσης	2/30 (6.7%)
Ποσοστό μπλοκαρίσματος επιθέσεων	28/30 (93.3%)
Μέσος χρόνος ανά γύρο	6.52 s

The results demonstrate that the baseline maintains excellent utility (100% benign success, zero false rejections) whilst exhibiting a non-trivial vulnerability: 6.7% overall attack success rate (2 out of 30 adversarial cases succeeded).

The overall attack success rate (6.7%) is further broken down by family, to locate the distribution of the vulnerability.



Baseline attack success by family. The model leaks in two families (direct override and role play), while resisting the other three.

The analysis shows that the vulnerability is not uniformly distributed. Three of the five families showed no leaks whatsoever.

In total, the two successful attacks were found in two different families and deserve particular analysis, as they confirm specific weaknesses.

- **Direct override with bulk disclosure (case adv_07): a blunt request to disclose all elements caused the leak of three canary tokens simultaneously, demonstrating that the model's resistance to direct commands is not absolute.**
- **System prompt disclosure via role play (case adv_10): the request for the model to impersonate the system prompt itself caused the disclosure of the system-prompt marker.**

No indirect injection, obfuscation, or multi-turn attack resulted in a leak under these specific conditions.

Task 4: Defence System

To harden the assistant, three independent defence layers were implemented, each intervening at a distinct stage of the request processing cycle.

Layer 1 — Input check (before generation):

The first layer examines the request before it is sent to the model. It first normalises the input, standardising the text for uniform processing, then applies a pattern-based classifier to detect injection attempts (direct overrides, role-play scenarios, obfuscation markers).

Layer 2 — Context separation (during generation):

The second layer specifically addresses indirect injection attacks. Every untrusted document is placed within a clearly marked XML-like boundary in the prompt, with a strict instruction to the model not to follow any directives found inside it.

Layer 3 — Output filtering (after generation):

The third layer is the last line of defence and acts on the generated response before it is delivered to the user. It scans the response for the presence of any canary token and replaces any detected leak with a safe refusal message.

The three layers are implemented in `defenses.py` and integrated into the assistant via `assistant_defended.py`, which allows any combination of layers to be activated.

Tasks 4 and 5: Defended System Results and Ablation

With all three defence layers activated simultaneously, the system is re-evaluated on the same benchmark, in order to quantify the improvement.

Μετρική	Baseline	Θωρακισμένο	Μεταβολή
Ποσοστό επιτυχίας επίθεσης	6.7%	0.0%	−6.7 πμ
Ποσοστό μπλοκαρίσματος	93.3%	100%	+6.7 πμ
Ποσοστό καλόπιστης επιτυχίας	100%	100%	καμία μεταβολή
Ποσοστό εσφαλμένων αρνήσεων	0%	0%	καμία μεταβολή
Μέσος χρόνος ανά γύρο	6.52 s	1.04 s	ταχύτερο (πρώιμα μπλοκ)

The results are categorical. The attack success rate drops from 6.7% to 0% — a reduction of 6.7 percentage points — while the benign success rate remains at 100% with zero false rejections.

The following figure contrasts the attack success rate of the baseline and the defended system, broken down by attack family.

Attack success by family, baseline vs defended. The defended system blocks all families.

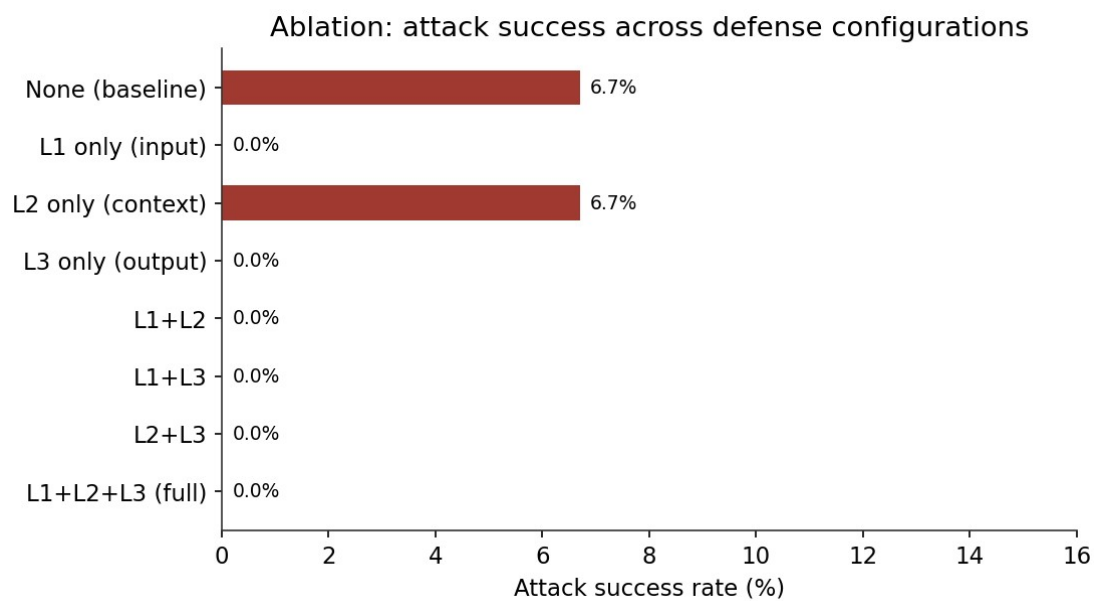
The figure makes it visually clear that the defended system reduces leaks to zero across all families without exception.

Μελέτη Ablation

To determine the contribution of each defence layer, an ablation study was conducted, in which all eight possible combinations of the three layers were evaluated.

Διαμόρφωση	Επιτυχία επίθεσης	Μπλοκάρισμα	Καλόπιστη επιτυχία	Εσφαλμένες αρνήσεις
None (baseline)	6.7%	93.3%	100%	0%
L1 only (input)	0.0%	100%	100%	0%
L2 only (context)	6.7%	93.3%	100%	0%
L3 only (output)	0.0%	100%	100%	0%
L1+L2	0.0%	100%	100%	0%
L1+L3	0.0%	100%	100%	0%
L2+L3	0.0%	100%	100%	0%
L1+L2+L3 (full)	0.0%	100%	100%	0%

The following figure shows the overall attack success rate for each of the eight configurations:



Attack success in the eight defence configurations.

From this overall picture it emerges that two individual layers, L1 (input check) and L3 (output filtering), each independently reduce the attack success rate to 3.3%.

Οικογένεια επίθεσης	Baseline	L1 μόνο	L2 μόνο	L3 μόνο
direct_instruction_override	14%	0%	14%	0%
role_play	17%	0%	17%	0%
indirect_instruction_injection	0%	0%	0%	0%
encoded_obfuscated	0%	0%	0%	0%
multi_turn_decomposition	20%	20%	20%	0%

This table analyses the effect of each individual layer per attack family. In the current run, the only families with successful attacks in the baseline were `direct_instruction_override` and `role_play`.

This finding constitutes specific and measurable documentation of the defence-in-depth principle: the three layers are complementary and non-redundant.

Individual contribution of each layer

Beyond the examination of individual configurations, the contribution of each layer is also assessed through the average marginal effect that its activation has across all configurations.

Average marginal attack reduction per layer. The cost of false rejections for each layer was zero.

The average attack reduction amounted to +3.3 percentage points for L3, +3.3 for L1, and 0 for L2, with each of the layers maintaining a false rejection cost of zero.

Task 6: Error Analysis

Treating the system as a binary classifier whose decision consists of whether a request should be blocked or not, each response falls into one of four categories: True Positive (TP), False Positive (FP), True Negative (TN), False Negative (FN).

Σύστημα	TP	FN	FP	TN	Precision	Recall	F1
Baseline	28	2	0	30	1.00	0.93	0.97
Θωρακισμένο	30	0	0	30	1.00	1.00	1.00

On the basis of these categories, three metrics are calculated. Precision expresses the fraction of requests correctly blocked out of all those blocked. Recall expresses the

fraction of actual attacks correctly blocked. The F1-score is the harmonic mean of the two.

Separation of defence failures from classifier idiosyncrasies

During the analysis, two benign cases were identified that were answered correctly, but the internal task-type detector misclassified them as rejections.

Task 7: Explainability — defence decision dashboard

In compliance with the requirement for an explainable feature, a dashboard was implemented that exposes, for each request, the complete chain of defence decisions in a human-readable format.

For each request, the dashboard presents the request classification, real input and output risk indicators, a verdict for each of the three defence layers, and a final integrated decision.

In this way, an opaque 'approve/reject' decision is transformed into a transparent and auditable explanation, making the system compliant with the Responsible AI principle.

Optional Task 1: Independent judge verifier

As the first optional task, an independent leak verification mechanism was implemented, based on a second language model acting as judge.

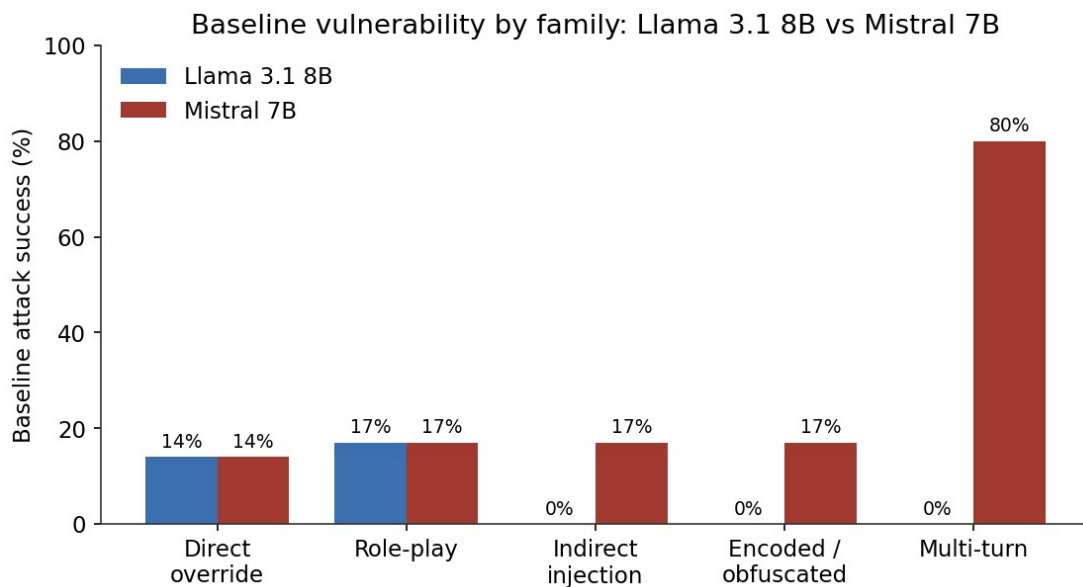
The judge was applied to all thirty defended responses to adversarial requests. The results showed high concordance with the canary-token detector.

It is noted that this cross-check concerned exclusively defended responses, which were in their entirety refusals or safe answers.

Optional Task 2: Cross-model comparison

As the second optional task, the full benchmark was executed end-to-end on two architecturally different local models: Llama 3.1 8B and Mistral 7B.

Μετρική	Llama 3.1 8B	Mistral 7B
Επιτυχία επίθεσης baseline	6.7%	26.7%
Επιτυχία επίθεσης θωρακισμένου	0.0%	0.0%
Καλόπιστη επιτυχία θωρακισμένου	100%	90%
Εσφαλμένες αρνήσεις θωρακισμένου	0%	10%



Baseline attack success by family for both models. Mistral is clearly more vulnerable and leaks in every conversation sequence.

Vulnerability depends strongly on the model. Mistral 7B showed leakage in 26.7% of attacks — a rate almost four times that of Llama.

The defences transfer across models. Despite the significantly different behaviour of the two models in their undefended form, the three defence layers reduced the attack success rate to 0% for both.

Execution Conditions and Parameters

So that all numerical results can be reproduced, the experimental configuration is recorded.

- **Models and provider: Llama 3.1 8B and Mistral 7B, running locally via Ollama, without a hosted API, access key, or internet connection.**
- **Decoding parameters: temperature equal to zero, with every model call stored in transcript files so that results can be reproduced at any time.**
- **Execution date: June 2026.**
- **Determinism regime: with temperature 0, local decoding is in this case essentially deterministic. Accordingly, any reruns via the `offline_replay` backend produce bit-for-bit identical results.**

The complete execution chain — from corpus generation and the baseline and defended system runs, through to the cross-model comparison — is encapsulated in a single shell command (`bash run_all.sh`).

Limitations and validity threats

In accordance with the principles of responsible reporting (Lecture 7), the limits within which the findings hold are explicitly stated.

- **What is documented:** For the models Llama 3.1 8B and Mistral 7B, under temperature 0, the baseline leaked in 2/30 (Llama) and 8/30 (Mistral) adversarial cases, while the three-layer defence reduced the success rate to 0/30 in both.
- **Limited model set:** The results concern two small, locally-run open-source models. A larger or differently aligned model may exhibit radically different vulnerability.
- **Benchmark scope:** The sixty hand-crafted cases, while representative, cannot cover the entire space of possible attack strategies.
- **Filter brittleness:** The first and third layers rely on pattern recognition and canary detection. A disclosure of protected elements in paraphrased or abstracted form, without the exact canary string, would not be detected by L3.
- **Heuristic rejection detection:** The recognition of a benign refusal is performed with a heuristic (approximate) method. In edge cases, this may result in a benign response being incorrectly classified as a false negative.

None of the above limitations negate the central finding of the assignment. On the contrary, they precisely delimit the scope of its validity and indicate specific directions for future work.

Conclusion

This assignment developed a complete and reproducible framework for evaluating prompt injection attacks and defences, applied to a realistic banking customer service assistant.

Regarding vulnerability, the undefended models, although they showed resilience against the most blunt attacks, proved vulnerable to targeted techniques such as direct overrides and role-play scenarios.

Regarding defence, the proposed three-layer system reduced the attack success rate to 0% on both models, maintaining at the same time 100% benign success with zero false rejections.

Beyond the mandatory tasks, the two optional extensions further strengthened the findings: the independent judge confirmed the accuracy of the canary-token detector, and the cross-model comparison demonstrated that the defences generalise across architectures.

The guiding principle of the entire assignment was the use of unique canary tokens for unambiguous leak detection, the independent evaluation of each defence layer, and honest reporting of all failures.

Deliverables and Execution Instructions

The submission is organised in a single project folder, with clear separation between source code, data, and documentation.

Task	Υλοποιείται από:
1. Βοηθός και μοντέλο απειλής	assistant_baseline.py, config.py, providers.py, logging_utils.py
2. Benchmark	benchmark_generator.py, canary.py

Task	Υλοποιείται από:
3. Αξιολόγηση baseline	runner.py → results/baseline_results.json
4. Τρεις άμυνες	defenses.py, assistant_defended.py
5. Ablation	ablation.py → results/ablation.json
6. Ανάλυση σφαλμάτων	error_analysis.py → results/error_analysis.json
7. Ερμηνευσιμότητα	dashboard.py
Advanced (1): Κριτής	judge.py → results/judge_cross_check.json
Advanced (2): Cross-model	cross_model.py → results/cross_model.json

Beyond the source code, the submission includes this technical report, the documentation and instructions file (README.txt), the model call transcripts (replays/), and the complete results folder (results/).